Invoking `GET cars_index_with_sample_settings/_settings` fetches the settings for this index.

- *Aliases*—Aliases are alternate names given to indexes. An alias can point to a single or multiple indexes. For example, an alias named `my_cars_aliases` could point to all car indexes. We can also execute queries on aliases as if we are running them on individual aliases. This code shows how to create an alias.

Listing 6.4 Creating an index with an alias

```
PUT cars_index_with_sample_alias
{
  "aliases": {
    "alias_for_cars_index_with_sample_alias": {}
  }
}
```

Creates the index

Declares the aliases object to configure the alias

The alias itself

We can issue `GET cars_index_with_sample_alias/_alias` to fetch the alias for this index.

When we explicitly create an index, we can set mappings, settings, and aliases up front. This way, the index is instantiated with all the required configurations in place. We can, of course, modify some of these configurations at runtime (we can tweak *closed* [non-operational] indexes, too). In the next section, we see how to set these configurations on explicitly created indexes.

6.2.2 Creating indexes explicitly

Indexes created implicitly are seldom ready for production configurations. Creating an index explicitly means we are expected to set custom configurations. We can direct Elasticsearch to configure an index with the required mappings and settings, as well as aliases, rather than depending on the defaults.

We already know that creating an index is easy: simply issue `PUT <index_name>`. This command creates a new index with the default configuration (similar to the index created when a document is indexed for the first time). For example, `PUT cars` creates a `cars` index, and issuing `GET cars` returns the index. Let's see how we can manage these indexes with custom configurations.

6.2.3 Indexes with custom settings

Every index can be instantiated with settings (default or custom) during creation. We can also change some settings when the index is still in operation. For this purpose, Elasticsearch exposes the `_settings` API to update the settings on a live index. However, as we mentioned, not all properties can be altered on a live index—only dynamic properties. That brings us to a brief discussion of the two types of index settings, static and dynamic:

- *Static settings*—Static settings are those that are (and can be) applied during index creation and cannot be changed while the index is in operation. These